

The use of session control in DAVIC to provide interactive multimedia services

R D Miles, P W Reece and L A Boon

This paper begins with an introduction to DAVIC, with particular reference to its objectives and plans. It then considers the split of the functional model into the three roles — service consumer system, delivery system and service provider system. Once the vertical split of the functional model has been considered the horizontal split into S1-S4 levels is covered. The paper then expands on the concepts of session and connection control through the use of the S2, S3 and S4 levels in the set-up, transfer and release of a user-to-network session and the end-to-end control of the user-to-user session. Finally, future plans/directions for DAVIC are considered, along with the adoption of Internet technologies within the DAVIC specifications.

1. Introduction to DAVIC

The Digital Audio-Visual Council (DAVIC) is a non-profit making association registered in Geneva. Its primary objective is to advance the success of emerging digital audiovisual applications by the timely production of specifications. Such specifications must be internationally agreed and based upon open interfaces and protocols that maximise interoperability across different countries and implementations. Initially the applications addressed by DAVIC include the broadcast and interactive type in which there is a significant high-quality digital audio/video component, such as video-on-demand (VoD) or home shopping.

The vision of the broadband future, where on-demand services and information are available at home, in the office and on the move, is now well established. Such a vision relies upon the integration of many networks and associated services, signalling and control. The current life cycle of multimedia applications and standards can be very short. Technical superiority does not necessarily guarantee universal adoption by all manufacturers, nor international standardisation. With this in mind, choosing the right building blocks for the broadband future vision is a risky business. Development of such a network is also extremely costly, which leads participants to seek collaboration in order to maximise interoperability and economies of scale. The goals of DAVIC are to identify, select, augment, develop and obtain the endorsement by formal standards bodies of specifications of interfaces, protocols and architectures of digital audiovisual applications and services. It was these goals that brought many of the key

players in the fields of entertainment, information technology and telecommunications to open international collaboration to produce the DAVIC specifications.

1.1 Membership

Membership of DAVIC is open to any corporation or individual firm, partnership, governmental body or international organisation. By December 1995, DAVIC had membership of over two hundred corporations representing more than twenty countries from all over the world and virtually all business communities with a stake in the emerging field of digital audiovisual applications and services.

1.2 The DAVIC 1.0 specification

The DAVIC 1.0 specification [1] has been developed by participating DAVIC members on the basis of submissions from both members and non-members in response to two calls for proposals which were issued respectively in October 1994 and March 1995. The public specification was finalised and published in January 1996.

The DAVIC 1.0 specifications have been developed within a multi-industry environment and must therefore be flexible enough to accommodate many distinct implementations based on individual requirements. The approach taken by DAVIC was to specify a set of tools, which lend themselves to flexible implementation rather than defining systems, which tend to be application specific. Compliance

with this principle leads to substantial benefits in terms of interoperability and availability of technology as the tool is applicable to a wider field of the technology. Examples of DAVIC tools include digital storage media, command and control (DSM-CC) based session control, and Moving Picture Experts Group (MPEG) encoded video streams.

2. The DAVIC system reference model

Figure 1 shows a general representation of the system addressed by the DAVIC specifications. It comprises five entities — the content provider system (CPS), the service provider system (SPS), and the service consumer system (SCS), with the three entities being connected by two delivery systems (DS), the first connecting the content provider to the service provider and the second the service provider to the service consumer. In principle, DAVIC specifications can address any part of the general system shown in Fig 1. The general DAVIC reference model is multi-dimensional in the sense that multiple SPSs may connect through many delivery systems to many SCSs. Each entity is composed of sub-system objects, which may or may not exist in a specific implementation of the architecture. The links between entities represent logical information flows. Similar flows may exist between sub-systems within an entity as well. The combination of entities, flows and sub-system objects can be used as a reference against which tools can be adopted and positioned within a particular implementation.

2.1 Reference points

Within the DAVIC system reference model, reference points define sets of logical interfaces through which peer objects transfer information. Within Fig 1, the following reference points are shown — A11 linking CPS and DS, A10 linking DS and SPS, A9 linking SPS and DS, and A1 linking DS and SCS.

A10 linking DS and SPS, A9 linking SPS and DS, and A1 linking DS and SCS.

2.2 System entities

The main DAVIC 1.0 system entities are the service provider system, delivery system and service consumer system.

- Service provider system

The service provider system is the primary means of accessing and manipulating the content that exists within, and flows through, a DAVIC system. The typical DAVIC 1.0 SPS could be a provider of a video-on-demand application.

- Delivery system

Within DAVIC the term delivery system has a very broad meaning. It refers to virtually any means to deliver information from one entity to another entity in order to support services defined by DAVIC. Entities involved are, among others, content provider systems, service provider systems and service consumer systems. Figure 1 depicts the general DAVIC model showing how the various entities are connected to the delivery system and through which reference points. Service providers can download content from the content providers and offer this content to end-consumers.

For DAVIC 1.0, only the delivery system between reference point A1 and A9 is considered. This delivery system is essential to bring the services offered by the service provider to the end consumers. In future versions of the DAVIC specifications, the delivery

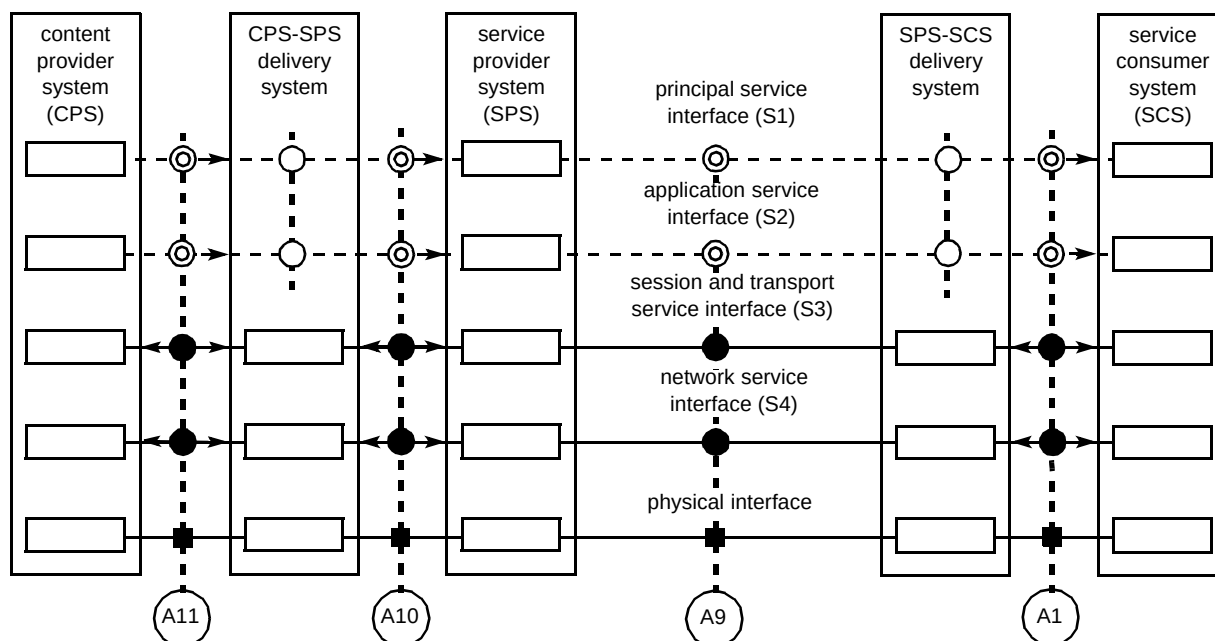


Fig 1 DAVIC system reference model.

system between the reference points A10 and A11 will be specified to ensure easy transfer of content between the content providers and the service providers.

Types of networked delivery systems range from telecommunications networks, such as co-axial cable or asynchronous transfer mode (ATM) networks, to satellite broadcast systems, such as direct broadcast satellite and terrestrial broadcast systems.

- **Service consumer system**

The SCS entity represents the combination of network interface unit (NIU), set-top unit (STU) and human or machine (e.g. video recorder) service consumer. The SCS contains functionality to permit the human or machine service consumer to interact with the service provider system and exert session control over the delivery system.

2.3 Sub-system objects

Internal to each of the main functional entities are additional sub-system objects that represent functionality within a particular element. These objects may be considered as functionality required to support the information flows between system entities. The location of sub-system objects within the main DAVIC functional entities is shown in Fig 2.

Service provider system objects

The service provider system comprises four core service elements — content service, application service, service gateway and stream service.

The content service element is responsible for the management of content, including loading and unloading content, between the server and the content provider as well as between the service elements.

The application service element is the collection of DSM-CC interface definitions on which all applications build.

The service gateway element is usually the first element the SCS observes and as such acts as a broker to the applications available within the SPS by providing the following functions:

- organising the service provider domain where services register (or install), making their existence known and de-registering when they are decommissioned — the service gateway adopts a name context interface, which provides a navigable name graph (analogous with a directory structure) to organise the service domain,

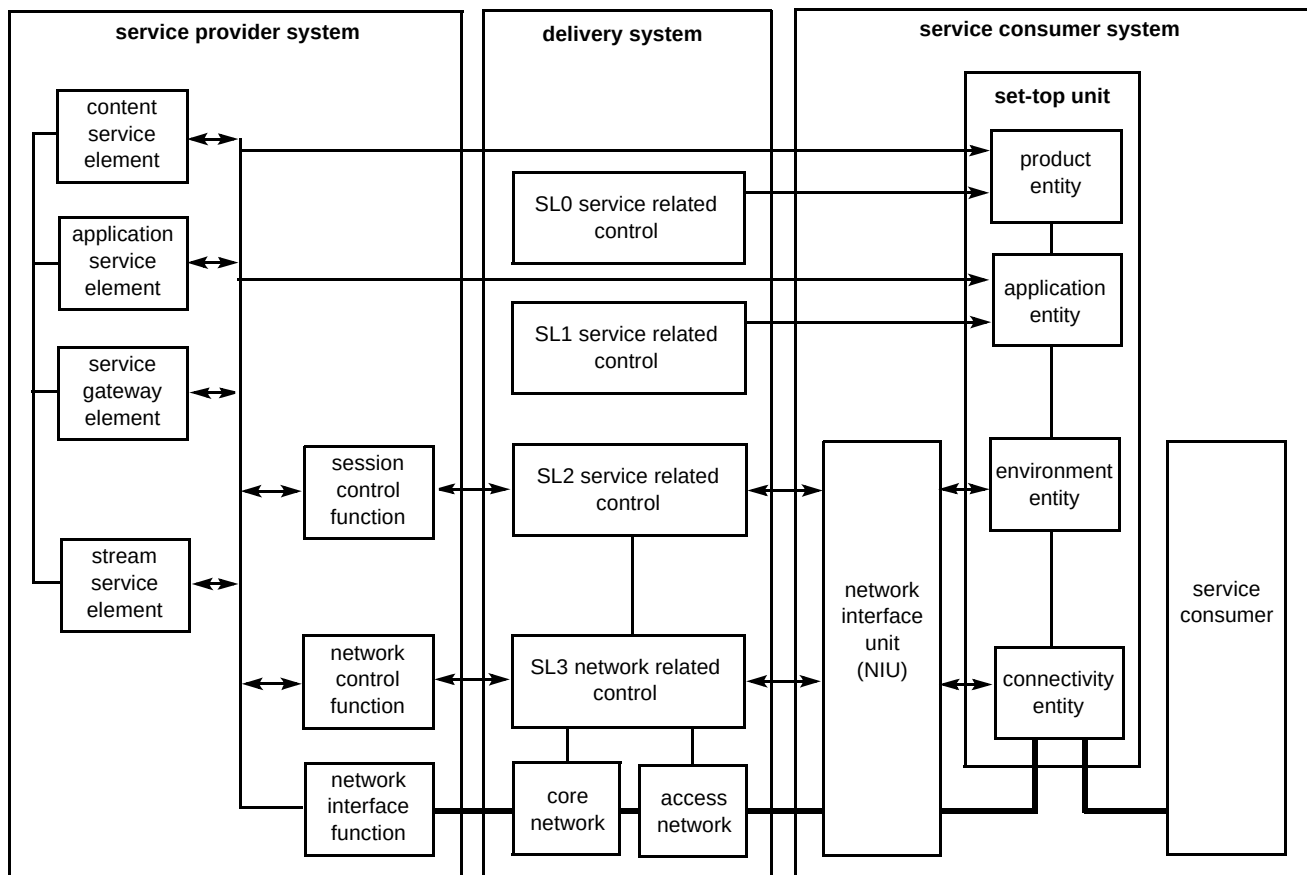


Fig 2 DAVIC functional entities.

- a means for the client to discover the existence of a service (e.g. a stream object for VoD) — this is accomplished by browsing the service domain at the S2 level, the objective often being to locate a specific service, known through the name bound to the name graph, i.e. an object of kind 'stream' and of ID 'batman',
- a means by which a client activates and deactivates an instance of a service — included in this process is session establishment and management through requests to the session control function (session object) leading to appropriate S3 and S4 messages.

The stream service element is a repository and source for streams. It provides the S2 interface through which the client device controls the media stream (play, pause, etc).

Delivery system objects

The SL0, SL1 and SL2 service-related control objects should be considered together as providing a brokering interface to the capabilities of the delivery system. One such use of these objects could be in providing DAVIC level one gateway functionality, whereby the end user can browse available DAVIC service provider systems in a manner analogous with a Yellow Pages or Yahoo service. In this particular case the name graph would represent a navigable structure of service gateways with which a session may be established, or transferred.

The network-related control entity provides control functions for network configuration, connection establishment and termination, and information routing. Within the delivery system, different sets of internal interfaces (not shown) may be used to encapsulate different core network and access network elements in order to provide interoperable, or compatible, delivery system services at the A1 and A9 reference points.

Service consumer system objects

The product entity is responsible for accepting content information and presenting the information to the end user of the SCS. Typically this role would be the decoding of multimedia and hypermedia expert group (MHEG) objects and MPEG content information and presentation to a television set or other end-user device.

The application entity represents and supports the end-user application running on the SCS. The DAVIC 1.1 specification has chosen a Java virtual machine to provide this role.

The environment entity is responsible for establishing and terminating the environment, or session, in which an application will operate. This session management may

make use of either the connectivity entity to perform connection control or a proxy signalling capability.

2.4 Information flows

The DAVIC system reference model, shown in Fig 1, contains four distinct information flows, which are labelled S1 to S4. Two types of flow exist — content information and control information. Figure 3 shows these information flows.

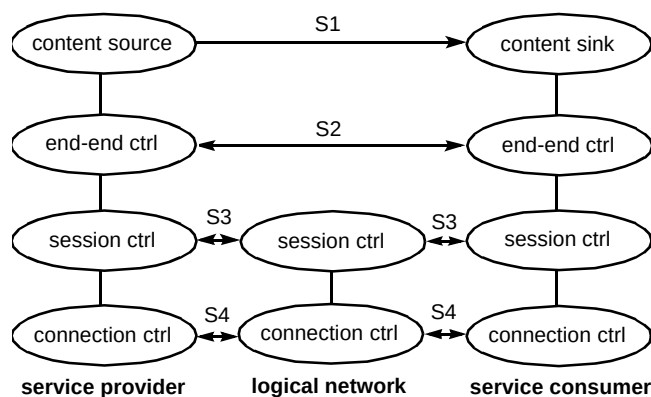


Fig 3 DAVIC information flows.

The S1 flow is unidirectional, between content source (SPS) and content sink (SCS) and used for the transportation of content information, such as MPEG encoded video. The flow is in the user plane and as such is transparent to any intermediate control objects through which it passes.

The S2 flow is bi-directional between end-to-end application control functionality. This is not an information flow in the traditional telecommunications sense, but a remote procedure call (RPC) plane based on an underlying Common Object Request Broker Architecture (CORBA). As such, information flows are of the form of remote procedure calls upon objects with defined methods and interfaces, rather than the exchange of sequences of messages between state machines.

The S3 flow is a bi-directional control information flow between end users, such as SPS and SCS, and the delivery system. Control information is exchanged in the form of sequences of messages in order to exert session control over the network resource in response to method invocations at the S2 level.

The S4 flow is a bi-directional control information flow between end users, such as SPS and SCS, and the delivery system. Control information is exchanged in the form of sequences of messages in order to exert connection control

over the network resource in response to session level requests at the S3 level.

3. DAVIC protocols

3.1 MPEG-2 (S1)

The tool chosen by DAVIC to provide this S1 information flow is the ISO MPEG encoding standard for video material. In particular, DAVIC specifies the use of MPEG-2 transport streams, which can comprise a number of individual MPEG video streams as well as private data (such as executable code).

The protocol stack defined by DAVIC for the delivery of S1 content information within an end-to-end ATM environment is shown in Fig 4.

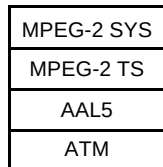


Fig 4 S1 protocol stack.

3.2 DSM-CC user-to-user and session objects (S2)

The S2 level is underpinned by a CORBA layer, on which DSM-CC user-to-user (U-U) objects within the service provider system may be manipulated via remote procedure calls. This section aims to give the reader a summary understanding of the operation of DSM-CC U-U, concentrating on its relationship with the session layer. The technical content is based upon the 12 July 1996 pre-editing release of the DSM-CC specification [2]. In order to give an overview of the DSM-CC U-U environment a truncated version of the DSM-CC system reference model is presented in Fig 5.

The session managers are not at the lowest layer of the system, as the original reference model has been truncated. Below the server session managers there are the ATM session, resource and control layers, but these are outside the scope of this section.

In summary, the DSM-CC U-U protocol is used to navigate to and then control a piece of content on a local or remote server.

Not all of the arrows shown in Fig 5 represent U-U interfaces. According to the DSM-CC specification, Table 1 represents those high-level interfaces implemented by the U-U protocol.

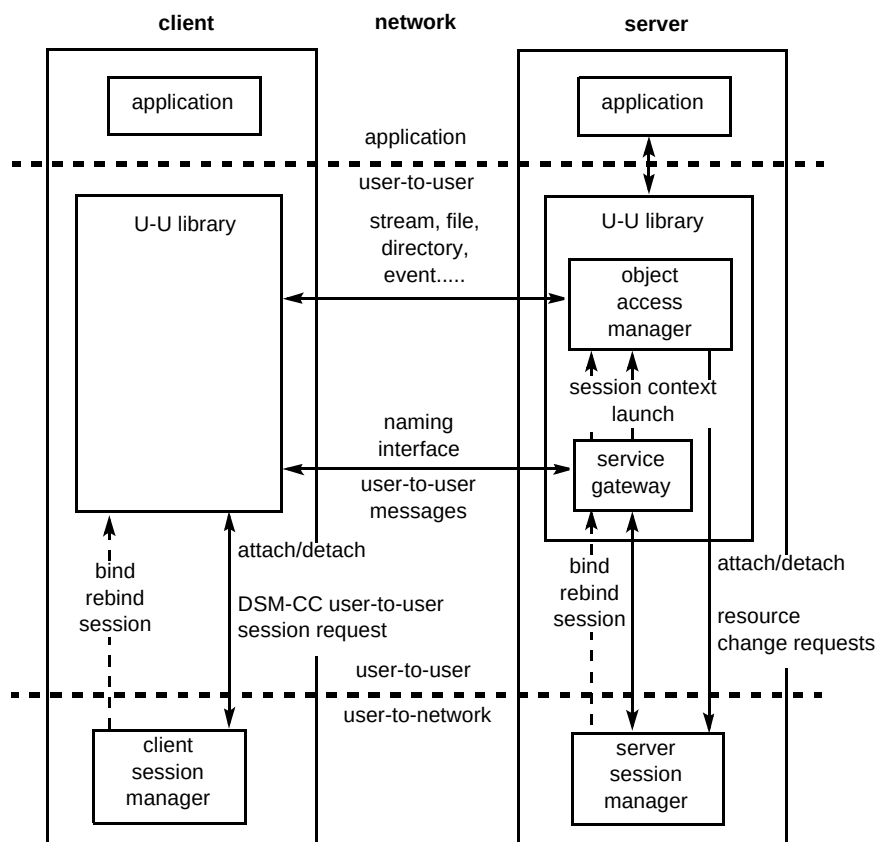


Fig 5 DSM-CC U-U system reference model.

Table 1 DSM-CC U-U high-level interfaces.

Peer 1	Peer 2
Client U-U library	Server service gateway
Client U-U library	Server object access
Client application	Client U-U libraries
Server application	Server U-U libraries

Each of the interfaces shown in Table 1 can be broken down into smaller component interfaces. The following parts of this section will describe these component interfaces. The total grouping of component interfaces is called the DSM-CC U-U, which can be seen in Fig 5. For simplicity only the core DSM-CC U-U libraries will be considered. Finally, the session interface is then examined in greater detail in section 3.2.2, showing the interaction of DSM-CC U-U and the session managers (client and server).

3.2.1 Core U-U libraries

To explain these interfaces fully would be a detailed exercise and would virtually involve a full replication of the DSM-CC specification [2]. A high-level understanding, though, is likely to help the reader comprehend the interaction between the DSM-CC U-U layer and the session managers (client and server).

Base interface

The base interface provides common operations for the deletion of DSM-CC object references and objects. All other core interfaces except access and directory interfaces inherit from this interface. Directory objects have their own closure and deletion procedures, for example:

- a client has finished using an object reference to access a remote service — the client uses the base interface to close it,
- a service object on a server is no longer required — the management of the server uses the base interface to delete it.

Access interface

The access interface provides common description and access control attributes, including size, version, date, lock and permission attributes, for example:

- a file object on a server contains important reference information on the services offered by that service provider — it is publicly available but should only be amended by the management of the server; the access interface is therefore used to place access restrictions on that object.

Stream primitives interface

Stream primitives are used to emulate VCR-like controls for manipulating MPEG continuous media streams, for example:

- a client has navigated their way to the film ‘*Batman*’ — in order for the film to be viewed in a controllable fashion, a recognisable interface must be provided, i.e. recognisable in terms of the client’s application being able to make use of it; the interface can then be used to take control of an MPEG stream with ‘play’, ‘stop’, ‘pause’, etc.

File interface

The file interface supplies two operations — file read and file write — for the manipulation of the content of files, for example:

- a text review of the film ‘*Men in Black*’ has been located by a client, and, in order to be able to view the contents of that file, a recognisable interface must be presented to the client’s application — this is similar to the case of the stream primitives interface.

Binding iterator interface

This interface provides the methods to return more bindings from a context (directory), if there are any, for example:

- a client has begun to list the contents of a directory and wishes to obtain the next name in that listing — methods provided by the binding iterator are used to obtain those subsequent names, if they exist.

Directory interface

The directory interface provides a general name space for binding names to services or data, for example:

- a service provider has a collection of service objects that must be ordered and associated in such a way that they are manageable and a client can gain easy access to them — the directory interface is used to perform this process of association and ordering,
- a client presented with a service provider faces a directory structure that contains the object references for service objects that are requested to be consumed — in order to navigate this structure the client uses the directory interface, which is also used to obtain the object reference that is required.

Session interface

The session interface enables a client to establish a session with a service gateway domain of services, for example:

- a client wishes to connect to a service provider — it is the operations of the session object that are invoked to perform the connection,
- a client wishes to resume watching the film ‘*Batman*’ — when they last left the service provider that they were using to view the film they suspended their session; by passing the correct parameters across the session interface the client application can reconnect to the service provider and the user can resume their viewing of the film.

Service gateway interface

This interface merely inherits from the directory and session interfaces, for example:

- a service provider’s management requires a single root or point of reference to which all of its services can be attached, and a single point of client access is also required — the service gateway interface provides this.

First interface

This interface enables an application to obtain its first objects, for example:

- a client wishes to obtain the object reference of the primary service that has been returned as the result of attaching to a service provider; this is not the object reference for the service gateway but rather a service further down the directory structure — it is the first interface that provides the necessary operation.

3.2.2 The session interface

The session interface consists of two operations ‘attach’ and ‘detach’. These are invoked across the U-U/U-N interface as can be seen in Fig 5. The session object itself resides on the U-N side of the interface.

The operation attach() takes three parameters — ServiceDomain, pathName and savedContext. ServiceDomain contains the NSAP address of the server that the client wishes to contact. pathName contains the path to a service that the user wishes to activate upon connection. This is a path through the directory structure of the service provider system. The DSM-CC specification states that either of these can be null if the other is set. savedContext is used to send information that is required by the server to resume an application from the point at which it was

suspended. DSM-CC does not state the format that the information takes.

The operation call attach() returns one parameter — ResolvedRefs. This contains the object reference for the service gateway which it has been connected and, if the pathName parameter was set, one or several object references for a first service.

The operation detach() takes one parameter — ASuspend — and returns one — savedContext. If ASuspend is set to TRUE, the state of the currently executing application will be returned in savedContext. It is this parameter that can then be passed in a subsequent attach() invocation.

DSM-CC states that attach() be invoked on the local session object and detach() be invoked on a remote session object. Detach() is invoked on the remote session object using the object reference for the service gateway to which it is connected. This object reference was originally obtained when attach() was invoked.

3.2.3 Summary

DSM-CC U-U uses session objects to obtain and resume connections to service providers and the services found on them. Local session objects are used to obtain initial connections. Remote session objects are used to close those connections. Once a session has been established the client application can use the U-U library to manipulate service objects on the service provider system with which it has established a session. In all of this it is implicit that the session objects form the bridge between the application and transport layers.

3.3 DSM-CC user-to-network (S3)

The protocol specific to the S3 flow is the ISO MPEG session control protocol, DSM-CC user-to-network (U-N). The protocol stack for DSM-CC U-N is given in Fig 6.

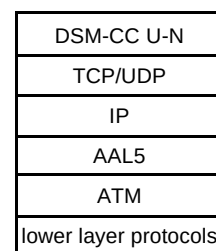


Fig 6 S3 protocol stack.

DAVIC supports a sub-set of the DSM-CC U-N messages. These are the messages involved in client session

set-up, server add and release resource, server/client, and network session release. DAVIC also specifies messages for session transfer should it be required in preference to subsequent release and set-up of the session to a new service domain.

3.4 Connection control — Q.2931 (S4)

The S4 flow is a bi-directional flow supporting call/connection control and resource control functions. The call/connection control functions consist of the capabilities to establish calls and connections in a B-ISDN network.

The protocol specific to the S4 flow is the International Telecommunications Union — Telecommunications (ITU-T) call/connection control protocol, Q.2931. This protocol supports the establishment and release of calls consisting of a single point-to-point connection in a broadband integrated services digital network (B-ISDN). The protocol stack for call/connection control is given in Fig 7.

Q.2931
Q.2130
Q.2110
AAL5
ATM
lower layer protocols

Fig 7 S4 protocol stack.

DAVIC supports a sub-set of Q.2931. These are call establishment (CALL PROCEEDING, CONNECT, CONNECT ACKNOWLEDGE and SETUP), call clearing (RELEASE and RELEASE COMPLETE), miscellaneous (STATUS and STATUS ENQUIRY) and global call reference messages (RESTART and RESTART ACKNOWLEDGE). In addition to the messages used, DAVIC also specifies the call/connection states and the information elements supported, and coding of specific information elements.

4. Session control

The DAVIC session is a means of encapsulating the set of network connections required for an instance of service usage. A session may require multiple connections at the S4 level, which are managed and controlled by a single S3 session. The application programming interface to the session is offered at the S2 level by the session object, within the end-user terminal (SCS and SPS). Thus, the session object exerts control over S3, which in turn subsequently controls S4.

The session control process leads to a sequence of information exchange both internal to a user system from

S2 to S3 and S3 to S4 message parameter mappings, and end-to-end between user and network across S3 and S4. The end result of session establishment is a set of S1 and S2 connections between SPS and SCS through which the user browses the service domain and retrieves application content.

4.1 S2 to S3 parameter mapping

When the operations session attach() and session detach() are invoked on a local session object their parameters must be carried to the remote system in S3 messages. This is because there is no S2 connection established to the entity with which they wish to establish a session. The parameters that need to be carried in the case of session attach() are ServiceDomain, pathName, savedContext and resolvedRefs. For session detach() they are ASuspend and savedContext. These are described in section 3.

All of the U-N session establishment and release messages contain a field for what is described in the DSM-CC specification as user-to-user data (uuData). It is into this field that the marshalled S2 parameters needing to be conveyed across the network are inserted. The uuData field is of variable length and so can cope with the variable length parameters that are passed.

4.2 S3 to S4 parameter mapping

In the flows shown in Figs 8, 9 and 10 it can be seen that on receipt of certain DSM-CC U-N messages the server must establish and release connections across the B-ISDN. These call/connections must be associated to the session and must be established at the required bandwidth and QoS.

It is not enough that the server knows the local call reference of the call/connections it has established in association with a session, the STB must also know with which session the call/connections are associated and this requires an end-to-end association. This association is made using a resource identifier, which is made up of a session ID and a resource number. This information is to be carried by the generic information transport (GIT) information element within Q.2931 messages. The GIT recommendation is currently under discussion within the ITU-T and the interim solution proposed uses the broadband higher layer identifier (BHL-I) information element.

The DSM-CC ResourceTypes is used to indicate that two connections are required through use of the AtmSvcConnection resource. However, there is no explicit indication of bandwidth and QoS for the S1 or S2 connections. In implementations, detailed within Reece et al [3] and elsewhere, this has been overcome by explicitly defining a bandwidth and QoS for S1 and S2 connections. The resource number field is used to identify the S1 and S2 connections.

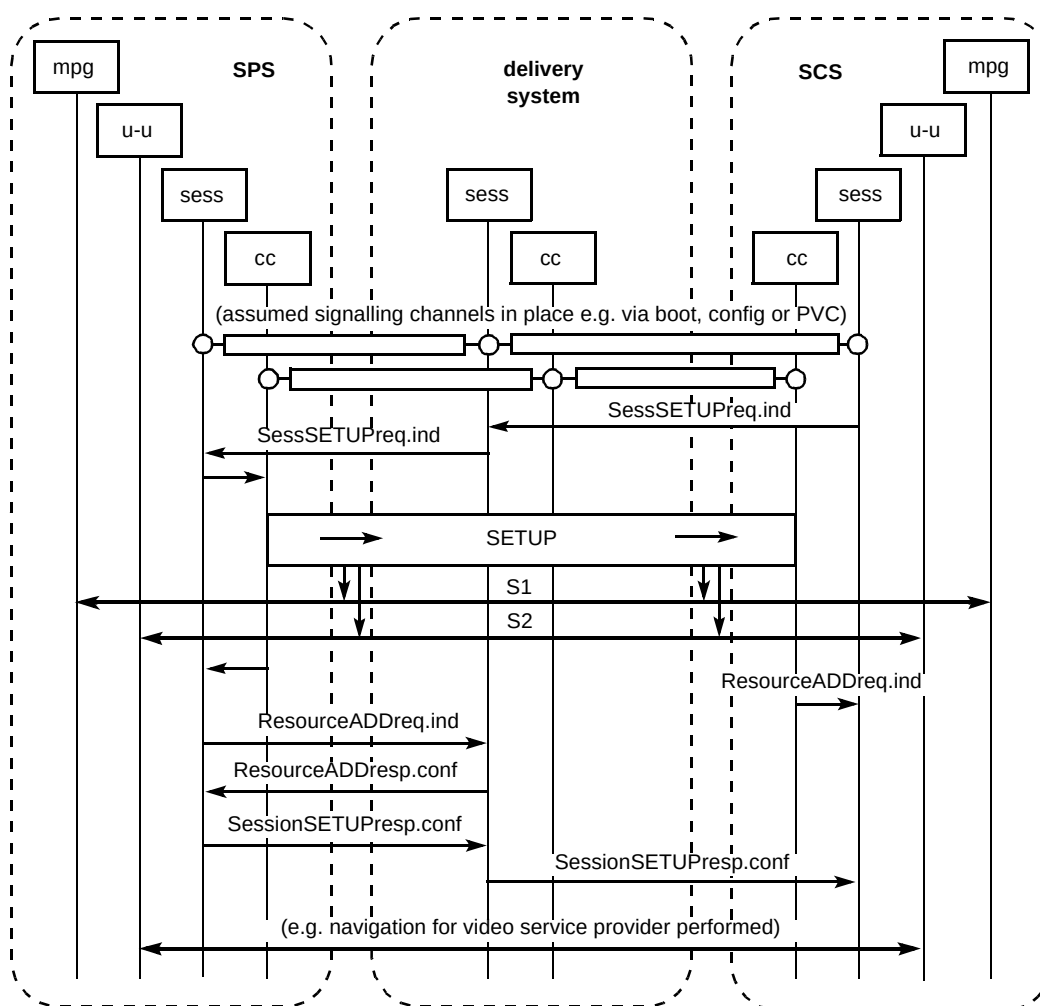


Fig 8 Session and call/connection set-up.

4.3 Dynamic operation

DAVIC use dynamic system modelling to specify how a DAVIC-compliant system behaves with respect to session and connection configuration. This takes the form of message flows between functional entities within a DAVIC system (see Fig 3).

A number of information flow scenarios are specified within the DAVIC recommendations — session and call/connection set-up, resource add, resource delete, transfer of session and call/connections between servers, service transfer, session and call/connection release. For many of these there are also further scenarios defined depending on whether they are SCS-initiated, network-initiated or server-initiated.

This paper focuses on what are envisaged to be the three main information flow scenarios — session and call/connection set-up (SCS-initiated), transfer of session and call/connections between servers (server-initiated), and

session and call/connection release (SCS-initiated). The scenarios below concentrate on the session and call/connection flows.

Session establishment

The information flows in Fig 8 show how an SCS makes a request to the SPS to establish a session. The requested session requires two connections across the network, a connection for the S1 flow and a second connection for the S2 flow. The SPS establishes these connections within the context of the session and confirmation that the requested session has been established is sent to the SCS.

Session transfer

The information flows in Fig 9 show how an SPS (SPS A) informs a second SPS (SPS B) that it has a session (and associated connections) it wishes to transfer from SPS A to SPS B. Due to the limitations of the call/connection control protocol it is not possible to simply transfer the connections to the new SPS. SPS B must set up two new connections in parallel to the two existing connections set up by SPS A.

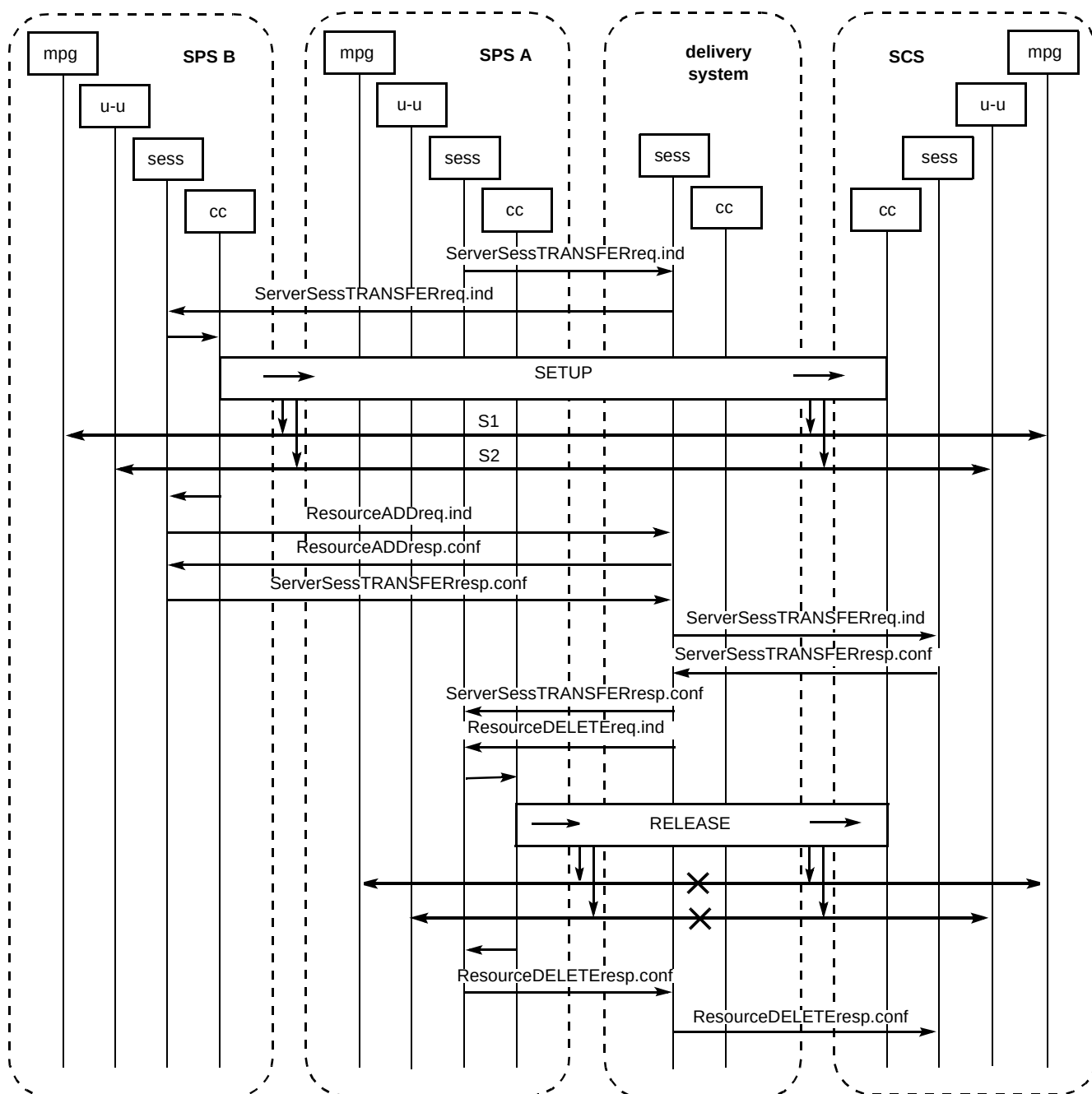


Fig 9 Transfer of session and call/connections between server.

Once these connections have been set up, SPS B can then notify the SCS that the session has been transferred (and the new connections can now be used). The SCS then notifies SPS A that the session has been transferred and the two connections established by SPS A are no longer required and are released.

Session release

The information flows in Fig 10 show how an SCS makes a request to the SPS to release a session. The release of the session requires that the two connections associated with the session are also released. The SPS releases these

connections and confirmation that the session has been released is sent to the SCS.

The information flow scenarios are further supported in the DAVIC recommendations by functional entity actions.

5. DAVIC development

5.1 Recent DAVIC history

Since the publication of the DAVIC 1.0 specification in January 1996 work has been ongoing to widen its scope beyond video-on-demand applications and access network technologies. This has been achieved through the addition of new tools and the publication of new versions of the

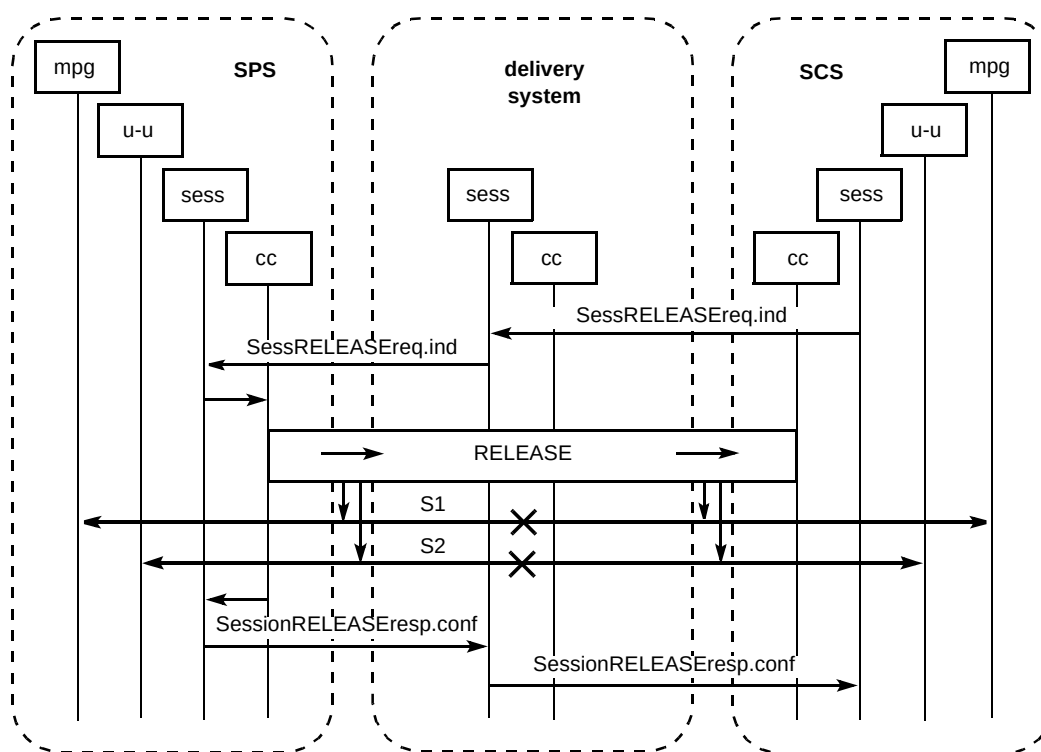


Fig 10 Session and call/connection release.

DAVIC specification. The DAVIC 1.1 specification, published in September 1996, primarily focused on enhancing the capabilities of the service consumer system, such as definitions of a network-independent interface, Internet access tools and a virtual machine based upon Java.

Version 1.2 of the DAVIC specification was released in December 1996 and focused upon improving the security of content and communications, enhanced Internet access and high-definition television (HDTV) quality video. DAVIC 1.2 provides a common specification that allows access from an SCS to high-quality audiovisual services and also to the Internet.

5.2 DAVIC and the internetworking protocol (IP)

The Internet is the main source for interactive information today. The broadcast television industry is already taking advantage of the Internet (broadcasters have Web sites, and links to Web pages are shown in many television programmes and commercials). The penetration and growth of IP-based applications has made it impossible for DAVIC to ignore IP. The strategy for the DAVIC 1.3 specifications is now strongly focused on the convergence of DAVIC and the Internet.

The World Wide Web provides a fantastically powerful and rich interactive environment based upon the hypertext mark-up language (HTML) and Java code. The one single thing preventing the Internet from becoming the means of delivering all interactive and broadcast services is its lack of throughput, or bandwidth. DAVIC delivery systems have the bandwidth and low latency to deliver very high quality

broadcast services as well as manageability, accountability and security included from the outset.

The ideal DAVIC 1.3 system will see the adoption of Internet protocols that will enable existing Internet use to continue unchanged, with the same equipment and protocols, but within the framework provided by DAVIC members through the specification process.

6. Conclusions

At the time that the DAVIC 1.0 specifications were drafted, VoD was considered to be a great revenue earner and, as such, many of the leading players in the IT and telecommunications field were racing to test and launch VoD equipment and broadband network technologies. It was on this wave of enthusiasm for VoD that the DAVIC specifications were created. From these initial steps towards a public VoD service, one thing became clear — VoD was not a get-rich-quick investment, requiring considerable up-front deployment of new technology, tight profit margins and a long time to wait before the service became profitable. Since the release of the 1.0 specification, enhanced digital broadcast has become the preferred medium to deliver interactive and broadcast services to the mass market. DAVIC has followed suit and is continuing to play a leading role in drafting specifications for enhanced digital broadcasting through alignment with and input to the European Digital Video Broadcasting (DVB) specification.

DAVIC was successful in raising the awareness of and interest in the use of session control as a mechanism to deliver interactive applications across a networked

environment. Session control is certainly defined in a more rigorous and flexible manner by the Telecommunications Information Network Architecture (TINA) specifications. The DAVIC session is, by comparison, naive, with user, communications and application attributes all tightly bound within the capabilities of DSM-CC. Inevitably the breadth of native DAVIC applications was limited by the original scope of DSM-CC, which was the client/server-based retrieval of video-based applications.

In the early days of DAVIC, few would have predicted the growth of Internet use that we see today. The Internet is now the leading platform for the provision of mass-market interactive information services. As with VoD in 1994, the challenge facing the IT world in 1998 is adding quality of service to IP-based applications in the Internet and on intranets. There are many competing technologies from virtually every ATM switch and IP router manufacturer, much as we saw with proprietary solutions for VoD in 1994. At present a winning technology has yet to emerge from this battle. DAVIC has now positioned itself with a systems integrator role whereby existing and emerging best-of-breed IP- standards and technologies can be chosen and integrated with existing DAVIC tools. The vision for DAVIC 1.3 is a network architecture that will allow existing IP-based applications to remain unchanged in their use of IP and the Internet, but enable them to make use of high-bandwidth, low-latency DAVIC infrastructure as required. Further reading on the activities of DAVIC is available from the DAVIC Web site [1].

References

- 1 DAVIC 1.0 specifications, <http://www.davic.org>
- 2 DSM-CC specifications, <ftp://playground.sun.com/pub/dsmcc/> (July 1996).
- 3 Reece P W et al: 'An early implementation of a DAVIC V1.0 system — use of dynamic connections for interactive multimedia services', BT Technol J, **16**, No 1, pp 114—126 (January 1998).



Richard Miles joined BT in 1986 as an apprentice at BT Laboratories. He then worked on optical access network technology until 1990, when he gained sponsorship to attend the University of Kent.

He graduated in 1993 with a BEng(Hons) in Electronic Engineering and joined the Intelligence Design and Performance unit, where he has been designing and prototyping systems for the delivery of broadband services.



Paul Reece graduated from Coventry Polytechnic with a BSc(Hons) in Mathematics in 1991 and joined the Broadband and Data Networks unit at BT Laboratories. He has been involved in a wide range of activities related to broadband signalling and network intelligence over the past six years.

He has recently moved to the Intelligence Design and Performance unit where he is leading a workpackage on network intelligence architectures and design.



Laurent Boon graduated from Durham University in 1995 with an Honours degree in Applied Physics and Computer Science.

He joined BT Laboratories in the same year where he has been working within the Intelligence Design and Performance unit on broadband network design and prototyping.